

PROJECTE FINAL
DESPLEGAMENT D'APLICACIONS WEB

El meu “Pas a Pas”:
“trafficlight-docker-challenge”

TASQUES 1 - 2

1) Fer un Fork del Projecte

[GitHub - hayk96/trafficlight-docker-challenge: :whale: A set of practice tasks for people who are new to Docker · GitHub](https://github.com/hayk96/trafficlight-docker-challenge)

2) Fer un git clone

```
C:\Users\HP\Documents\DAW 2N\DESPLÉGAMENT D'APLICACIONS WEB\PFAV3>git clone https://github.com/ssabir94/trafficlight-docker-challenge.git
Cloning into 'trafficlight-docker-challenge'...
remote: Enumerating objects: 63, done.
remote: Counting objects: 100% (35/35), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 63 (delta 21), reused 10 (delta 10), pack-reused 28 (from 1)
Receiving objects: 100% (63/63), 13.11 KiB | 671.00 KiB/s, done.
Resolving deltas: 100% (26/26), done.

C:\Users\HP\Documents\DAW 2N\DESPLÉGAMENT D'APLICACIONS WEB\PFAV3>cd trafficlight-docker-challenge

C:\Users\HP\Documents\DAW 2N\DESPLÉGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 54DA-D363

Directorio de C:\Users\HP\Documents\DAW 2N\DESPLÉGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge

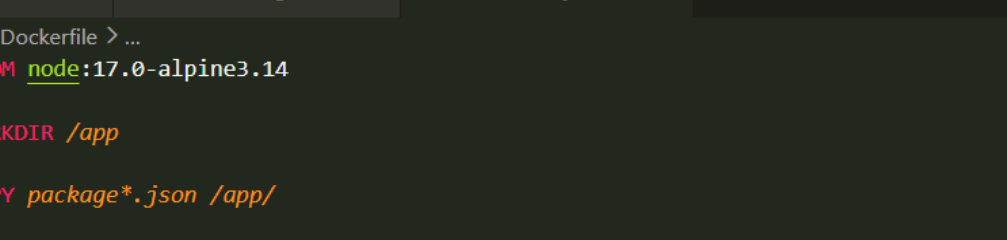
10/05/2026  10:46  <DIR>      .
10/05/2026  10:46  <DIR>      ..
10/05/2026  10:46                9 CODEOWNERS
10/05/2026  10:46  <DIR>      green
10/05/2026  10:46                1.090 LICENSE
10/05/2026  10:46                8.435 README.md
10/05/2026  10:46  <DIR>      red
10/05/2026  10:46  <DIR>      yellow
10/05/2026  10:46                48 _config.yml
              4 archivos              9.582 bytes
```

3) Veure la versió Docker:

```
C:\Users\HP\Documents\DAW 2N\DESPLÉGAMENT D'APLICACIONS
WEB\PFAV3\trafficlight-docker-challenge>docker --version
Docker version 28.4.0, build d8eb465
```

```
C:\Users\HP\Documents\DAW 2N\DESPLÉGAMENT D'APLICACIONS
WEB\PFAV3\trafficlight-docker-challenge>docker compose version
Docker Compose version v2.39.4-desktop.1
```

4) Copio el mateix a tots els DockerFile (segons indica github)



The screenshot shows a Dockerfile editor with three tabs: "Dockerfile red M", "Dockerfile green M", and "Dockerfile yellow M X". The active tab is "Dockerfile yellow M X". The editor displays a multi-stage Dockerfile with the following instructions:

```
1 FROM node:17.0-alpine3.14
2
3 WORKDIR /app
4
5 COPY package*.json /app/
6
7 RUN apk add --no-cache npm=7.17.0-r0 && npm install
8
9 COPY app.js /app/
10
11 RUN mkdir -p /app/views
12
13 COPY index.pug favicon.ico /app/views/
14
15 EXPOSE 80
16
17 CMD ["node", "app.js"]
```

The editor interface includes a line number column on the left, a command prompt on the right, and a status bar at the bottom.

5) Inicialitzo Docker:

```
C:\Users\HP\Documents\DAW_2N\DESPLÉGAMENT D' APLICACIONS WEB\PFVAV3\trafficlight-docker-challenge>docker build -t trafficlight/red:v1.0 ./red
```

	docker:desktop-linux
[+] Building 11.1s (4/11)	
=> [internal] load build definition from Dockerfile	0.1s
=> => transferring dockerfile: 292B	0.0s
=> [internal] load metadata for docker.io/library/node:17.0-alpine3.14	2.1s
=> [internal] load .dockerignore	0.1s
=> => transferring context: 2B	0.0s
=> [1/7] FROM docker.io/library/node:17.0-alpine3.14@sha256:959c4fc79a753b8b797c4fc9da967c7a81b4a3aff93d484dfe00092bf9fd584	8.7s
=> => resolve docker.io/library/node:17.0-alpine3.14@sha256:959c4fc79a753b8b797c4fc9da967c7a81b4a3aff93d484dfe00092bf9fd584	0.1s
=> => sha256:c55a18c6df4f9256a45b671793dd4c250fa9ec710c6807db64b3bb64d02f253c 455B / 455B	0.3s
=> => sha256:2380bcf66246f6a1af510aa8fd1cfe2e96530a3d28a5ab62c0f1ce8a9de341cd 2.35MB / 2.35MB	1.5s
=> => sha256:5917162e2ae60016f6014a42bad734fb5efc3310be5efec9fe33c0d2f3e7d78 39.85MB / 45.50MB	8.4s
=> => sha256:a0d0a0d46f8b52473982a3c466318f479767577551a53ffc9074c9fa7035982e 2.81MB / 2.81MB	1.7s
=> => extracting sha256:a0d0a0d46f8b52473982a3c466318f479767577551a53ffc9074c9fa7035982e	0.6s
=> [internal] load build context	0.2s
=> => transferring context: 4.20kB	0.1s

Faig el mateix amb els altres dos colors.

6) Comprovar imatges creades:

Images [Give feedback](#)

Local My Hub

0 Bytes / 5.76 GB in use 5 images Last refresh: 4 minutes ago

Search

☐	Name	Tag	Image ID	Created	Size	Actions
☐	mysql	8.0	68bee0989e56	3 months ago	1.06 GB	▶ ⋮ 🗑
☐	mysql	5.7	4bc6bc963e6d	2 years ago	688.73 MB	▶ ⋮ 🗑
☐	trafficlight/red	v1.0	53804610b0df	3 minutes ago	345.79 MB	▶ ⋮ 🗑
☐	trafficlight/yellow	v1.0	6c0967161647	1 minute ago	345.79 MB	▶ ⋮ 🗑
☐	trafficlight/green	v1.0	e326f10f0d40	49 seconds ago	345.79 MB	▶ ⋮ 🗑

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>docker images
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
trafficlight/green   v1.0        e326f10f0d40 2 minutes ago 346MB
trafficlight/yellow  v1.0        6c0967161647 2 minutes ago 346MB
trafficlight/red     v1.0        53804610b0df 4 minutes ago 346MB
```

7) Crear xarxa a Docker:

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>docker network create
--subnet=172.20.0.0/16 --gateway=172.20.0.1 traffic-light
58fbf55d55c1e7e0f9c966d1daa47c10d449e38337b0dc1ab0f17f5bf87fc395
```

8) Arrencar primer contenidor:

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>docker run -d --name red-app
--network traffic-light trafficlight/red:v1.0
0dfac561905ae0877bd72081152a95492ad0d164da769eff7747d283ea1bd23b
```

Comprovo que és viu:

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
0dfac561905a   trafficlight/red:v1.0  "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes  80/tcp       red-app
```

Containers [Give feedback](#)

Container CPU usage ⓘ

0.00% / 400% (4 CPUs available)

Container memory usage ⓘ

13.36MB / 3.68GB

[Show charts](#)

Q Search



☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Memory (%)	Disk read/v	Actions
☐	● red-app	0dfac561905a	trafficlight/		0%	13.36MB / 3.76Gi	0.35%	41.7MB / 0	☐ ☐ ☐

9) Arrencar la resta de contenidors:

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>docker run -d --name yellow-app --network traffic-light trafficlight/yellow:v1.0
2059a44ea770c1df86883a5c1f627e83f61b2edb05bec1dea03439a29ad7c01e

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>docker run -d --name green-app --network traffic-light trafficlight/green:v1.0
e8fc25efc860dd3f79352cad92c6f9fb8e1b52d6d402e0306488a1b7b20939a6
```

Containers [Give feedback](#)

Container CPU usage ⓘ

0.00% / 400% (4 CPUs available)

Container memory usage ⓘ

42.37MB / 3.68GB

[Show charts](#)

Q Search



☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Memory (%)	Disk read/v	Actions
☐	● red-app	0dfac561905a	trafficlight/		0%	13.52MB / 3.76Gi	0.35%	41.7MB / 0	☐ ☐ ☐
☐	● yellow-app	2059a44ea770	trafficlight/		0%	15.73MB / 3.76Gi	0.41%	4.24MB / 0	☐ ☐ ☐
☐	● green-app	e8fc25efc860	trafficlight/		0%	13.12MB / 3.76Gi	0.34%	4.1KB / 0B	☐ ☐ ☐

10) Creació de carpeta nginx + -> conf.d

11) Creació arxiu (dins carpeta conf.d): default.conf

12) Modificar arxiu:

```
default.conf U X
nginx > conf.d > default.conf
1  server {
2      listen 3000;
3
4      location / {
5          proxy_pass http://red-app:80;
6      }
7  }
8
9  server {
10     listen 4000;
11
12     location / {
13         proxy_pass http://yellow-app:80;
14     }
15 }
16
17 server {
18     listen 5000;
19
20     location / {
21         proxy_pass http://green-app:80;
22     }
23 }
```

13) Descarrega de nginx

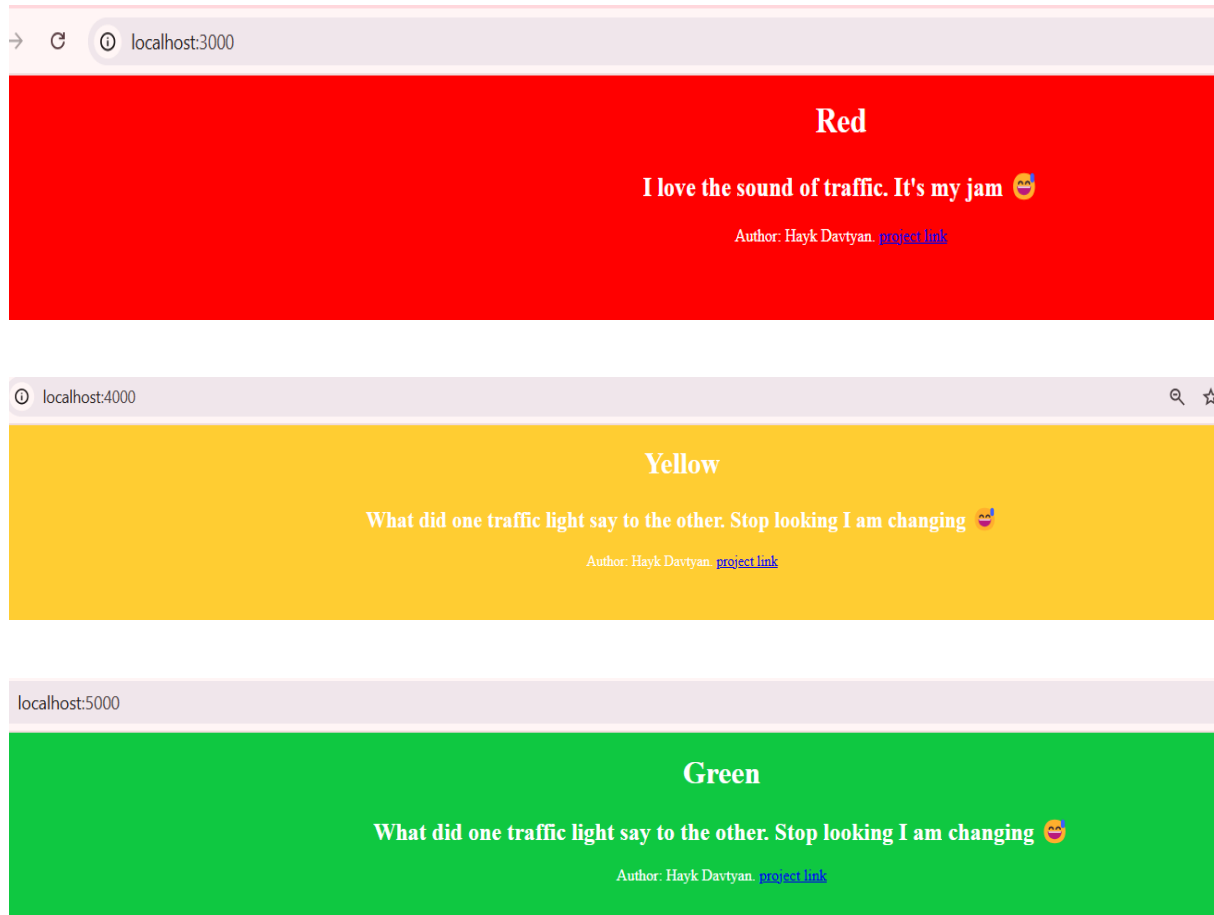
```
docker pull nginx:1.21
```

```
C:\Users\HP\Documents\DAW 2N\DESPLLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>docker pull nginx:1.21
1.21: Pulling from library/nginx
42c077c10790: Pull complete
62c70f376f6a: Pull complete
75a963e94de0: Pull complete
7b1fab684d70: Pull complete
db24d06d5af4: Pull complete
915cc9bd79c2: Pull complete
Digest: sha256:2bcabc23b45489fb0885d69a06ba1d648aeda973fae7bb981bafbb884165e514
Status: Downloaded newer image for nginx:1.21
docker.io/library/nginx:1.21
C:\Users\HP\Documents\DAW 2N\DESPLLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>
```

14) Arrencar nginx-proxy

```
docker run -d --name nginx-proxy --network traffic-light -p 3000:3000 -p 4000:4000
-p 5000:5000 -v "%cd%/nginx/conf.d:/etc/nginx/conf.d" nginx:1.21
```

15) Provar una a una



16) Comprovar si realment surten els canvis amb git status:

```
C:\Users\HP\Documents\DAW 2N\DESPLGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   green/Dockerfile
        modified:   red/Dockerfile
        modified:   yellow/Dockerfile

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        nginx/

no changes added to commit (use "git add" and/or "git commit -a")

C:\Users\HP\Documents\DAW 2N\DESPLGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>
```

17) Pujar els canvis de Task 1 i Task 2 a GitHub:

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>git add .

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>git commit -m "Task 1 and Task 2 completed"
[main 6815bc9] Task 1 and Task 2 completed
 4 files changed, 50 insertions(+), 36 deletions(-)
 create mode 100644 nginx/conf.d/default.conf

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>git push
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 4 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 1.06 KiB | 180.00 KiB/s, done.
Total 9 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/ssabir94/trafficlight-docker-challenge.git
   9d839c8..6815bc9  main -> main
```


TASCA 3

1) Parar els contenidors:

```
docker stop nginx-proxy red-app yellow-app green-app
```

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>docker stop nginx-proxy red-app yellow-app green-app
nginx-proxy
red-app
yellow-app
green-app
```

2) Esborrar contenidors:

```
docker rm nginx-proxy red-app yellow-app green-app
```

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>docker rm nginx-proxy red-app yellow-app green-app
nginx-proxy
red-app
yellow-app
green-app
```

3) Creació d'arxiu a arrel del projecte:

```
docker-compose.yml
```

4) Edició del DockerCompose

```
services:

  red-app:
    image: trafficlight/red:v1.0
    container_name: red-app
    networks:
      - traffic-light

  yellow-app:
    image: trafficlight/yellow:v1.0
    container_name: yellow-app
    networks:
      - traffic-light

  green-app:
    image: trafficlight/green:v1.0
    container_name: green-app
    networks:
      - traffic-light

  nginx-proxy:
    image: nginx:1.21
    container_name: nginx-proxy
    ports:
      - "3000:3000"
      - "4000:4000"
      - "5000:5000"
    volumes:
      - ./nginx/conf.d/default.conf:/etc/nginx/conf.d/default.conf
    networks:
      - traffic-light
    depends_on:
      - red-app
      - yellow-app
      - green-app

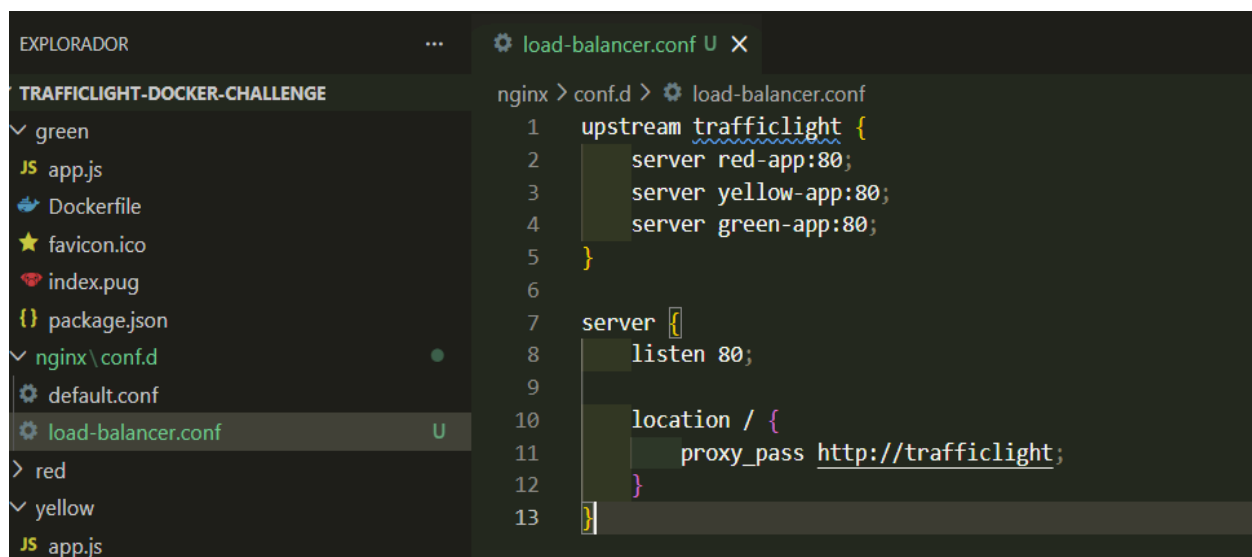
networks:
  traffic-light:
    external: true
```

5) Afegir nginx-load-balancer

```
Run Service
nginx-load-balancer:
  image: nginx:1.21
  container_name: nginx-load-balancer
  ports:
    - "3000:80"
  volumes:
    - ./nginx/conf.d:/etc/nginx/conf.d
    - /var/log/nginx:/var/log/nginx
  networks:
    - traffic-light
networks:
  traffic-light:
    external: true
```

6) Canviar Nginx a Mode Load Balancer

Com que el exercici demana que tinguem totes les configuracions, crearé un altre arxiu:



```
load-balancer.conf
nginx > conf.d > load-balancer.conf
1 upstream trafficlight {
2     server red-app:80;
3     server yellow-app:80;
4     server green-app:80;
5 }
6
7 server {
8     listen 80;
9
10    location / {
11        proxy_pass http://trafficlight;
12    }
13 }
```

Així segueix complint els requisits de l'exercici.

7) Seguint amb els requeriments, crearé un altre arxiu:

docker-compose-load-balancer.yml

```
services:

  red-app:
    image: trafficlight/red:v1.0
    container_name: red-app
    networks:
      - traffic-light

  yellow-app:
    image: trafficlight/yellow:v1.0
    container_name: yellow-app
    networks:
      - traffic-light

  green-app:
    image: trafficlight/green:v1.0
    container_name: green-app
    networks:
      - traffic-light

  nginx-load-balancer:
    image: nginx:1.21
    container_name: nginx-load-balancer
    ports:
      - "3000:80"
    volumes:
      -
        ./nginx/conf.d/load-balancer.conf:/etc/nginx/conf.d/default.conf
      - /var/log/nginx:/var/log/nginx
    networks:
      - traffic-light
    depends_on:
      - red-app
      - yellow-app
      - green-app

networks:
  traffic-light:
    external: true
```

8) Provar load balancer

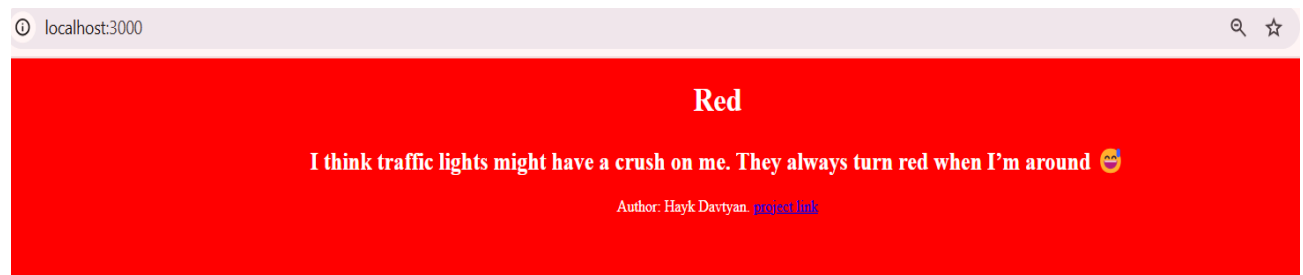
Assegurar de que no queda res aixecat:

docker compose down

docker compose -f docker-compose-load-balancer.yml up -d

```
C:\Users\HP\Documents\DAW 2N\DESPLIEGAMIENTO D'APLICACIONES WEB\PF3\trafficlight-docker-challenge>docker compose -f docker-comp
ose-load-balancer.yml up -d
[+] Running 4/4
 ✓ Container red-app          Started                               2.2s
 ✓ Container green-app        Started                               2.2s
 ✓ Container yellow-app        Started                               2.2s
 ✓ Container nginx-load-balancer Started                               3.1s
```

9) Fer la comprovació de l'alternança:



10) Creació carpeta `auth + → .htpasswd`

11) Copiar credencial dintre d'arxiu

12) Afegir canvis a `load-balancer.conf`

```
upstream trafficlight {
    server red-app:80;
    server yellow-app:80;
    server green-app:80;
}

server {
    listen 80;

    auth_basic "Restricted Access";
    auth_basic_user_file /etc/nginx/.htpasswd;

    location / {
        proxy_pass http://trafficlight;
    }
}
```

13) Modificar volums de `docker-compose-load-balancer.yml`

```
volumes:
  - ./nginx/conf.d/load-balancer.conf:/etc/nginx/conf.d/default.conf
  - ./auth/.htpasswd:/etc/nginx/.htpasswd
  - /var/log/nginx:/var/log/nginx
```

14) Reiniciar load-balancer

`docker compose -f docker-compose-load-balancer.yml down`

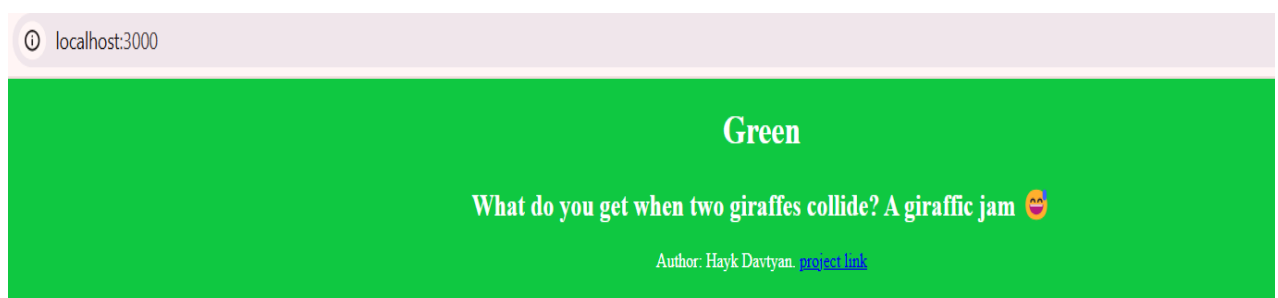
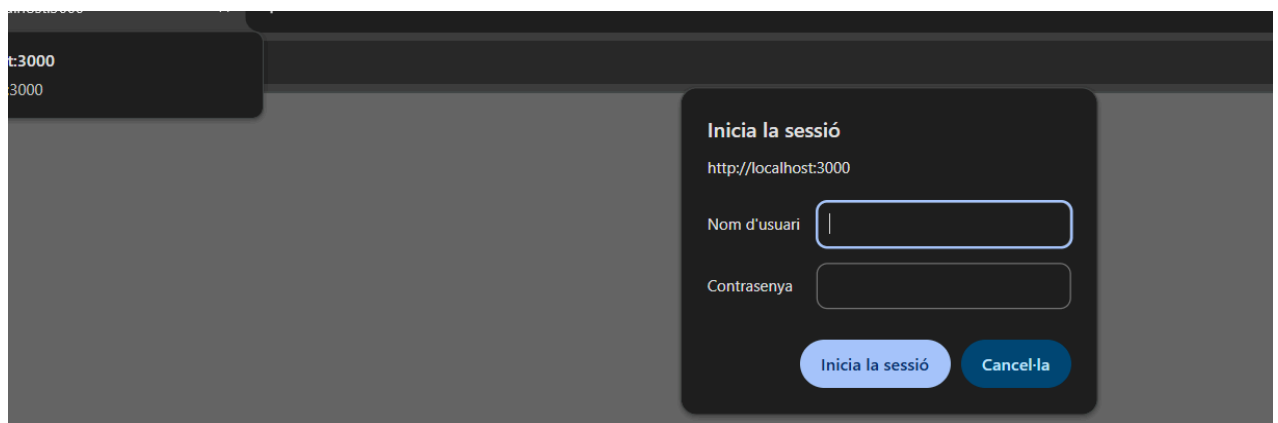
```
C:\Users\HP\Documents\DAW 2N\DESPLIEGAMENT D'APLICACIONS WEB\PF3\trafficlight-docker-challenge>docker compose -f docker-comp
ose-load-balancer.yml down
[+] Running 4/4
✔ Container nginx-load-balancer Removed 0.8s
✔ Container red-app Removed 11.4s
✔ Container yellow-app Removed 11.2s
✔ Container green-app Removed 11.1s
```

15) Aixecar-ho:

```
docker compose -f docker-compose-load-balancer.yml up -d
```

```
C:\Users\HP\Documents\DAW 2N\DESPLIEGAMENT D'APLICACIONS WEB\PFAV3\trafficlight-docker-challenge>docker compose -f docker-comp
ose-load-balancer.yml up -d
[+] Running 4/4
✔ Container yellow-app      Started      1.7s
✔ Container red-app         Started      1.8s
✔ Container green-app       Started      1.8s
✔ Container nginx-load-balancer Started      2.5s
```

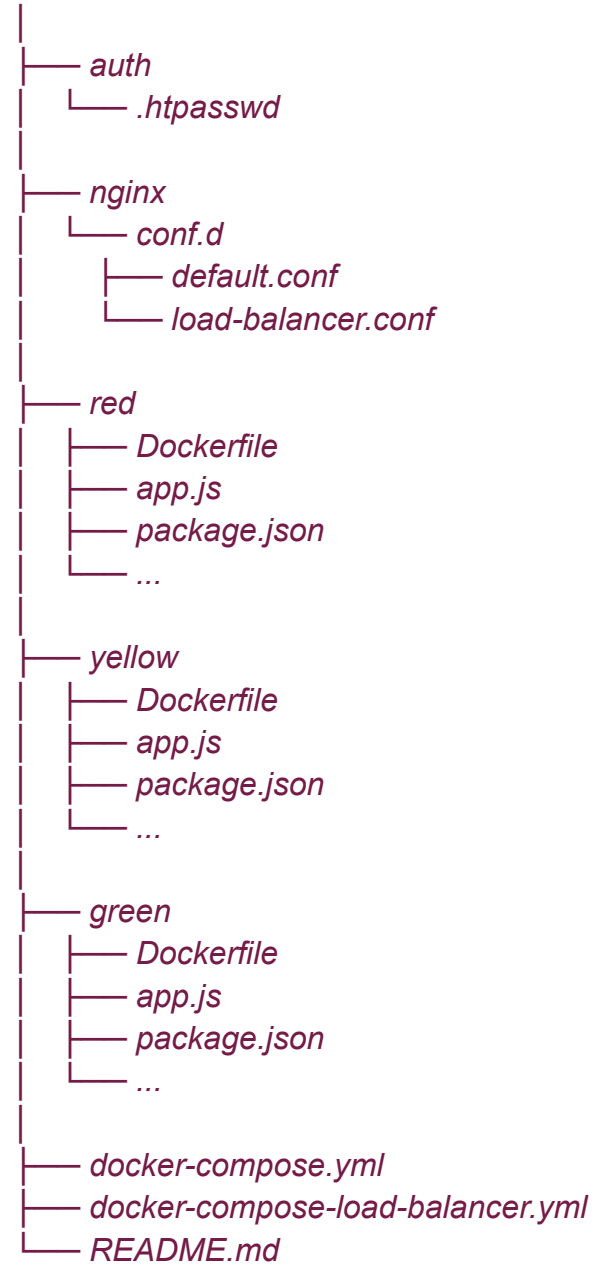
16) Obrir <http://localhost:3000>



Tot funciona genial :)

ESTRUCTURA FINAL PROJECTE

trafficlight-docker-challenge



Execució de la Versió Proxy Invers

Per executar la primera part del projecte (proxy invers):

```
docker compose up -d
```

URLs disponibles:

```
http://localhost:3000 → red-app
```

```
http://localhost:4000 → yellow-app
```

```
http://localhost:5000 → green-app
```

Aquesta versió utilitza nginx com a proxy invers, enviant cada port a una aplicació diferent.

Execució de la Versió Load Balancer

Per executar la segona part del projecte (balancejador de càrrega):

```
docker compose -f docker-compose-load-balancer.yml up -d
```

URL disponible:

```
http://localhost:3000
```

En refrescar la pàgina, nginx reparteix les peticions entre:

- *red-app*
- *yellow-app*
- *green-app*

Aquesta configuració utilitza Nginx HTTP Load Balancing.

Autenticació Bàsica

A la versió Load Balancer s'ha implementat autenticació bàsica amb nginx (**Basic Authentication**).

Credencials utilitzades:

```
Usuari: admin
```

```
Contrasenya: admin123
```

Això restringeix l'accés a les aplicacions web abans d'entrar al sistema.

Conclusió Final

Durant aquest projecte he pogut treballar amb Docker, Docker Compose i nginx per desplegar diferents aplicacions web dins de contenidors i comunicar-les entre elles mitjançant una xarxa interna.

A la primera part del projecte he creat els Dockerfiles de les tres aplicacions (**red**, **yellow** i **green**) seguint les especificacions del challenge. Totes les aplicacions comparteixen la mateixa estructura i utilitzen la mateixa imatge base (**node:17.0-alpine3.14**) per mantenir coherència entre contenidors i facilitar el manteniment del projecte.

Posteriorment he creat una xarxa Docker pròpia anomenada **traffic-light**, fet que ha permès que els contenidors poguessin comunicar-se entre ells utilitzant els noms dels serveis (**red-app**, **yellow-app**, **green-app**) en lloc d'adreces IP manuals. Aquesta decisió simplifica molt la configuració i fa el projecte més escalable.

Per implementar el proxy invers amb nginx he decidit separar la configuració dins de la carpeta **nginx/conf.d**, de manera que la configuració es pogués gestionar directament des del sistema host utilitzant volums Docker. Això permet modificar la configuració de nginx sense reconstruir contenidors.

En la segona part del projecte he implementat un sistema de balanceig de càrrega (**load balancing**) amb nginx utilitzant la directiva **upstream**. Aquesta configuració reparteix les peticions entre les tres aplicacions web de manera automàtica. Per comprovar-ho, he realitzat múltiples refreshs del navegador observant com es mostraven alternativament les aplicacions **red**, **yellow** i **green**.

Una de les decisions importants del projecte ha estat separar les dues configuracions principals:

- una configuració per al proxy invers
- una configuració per al balancejador de càrrega

Per aquest motiu he creat:

- **docker-compose.yml**
- **docker-compose-load-balancer.yml**

i també dos arxius de configuració nginx:

- `default.conf`
- `load-balancer.conf`

D'aquesta manera el projecte compleix millor els requeriments de l'enunciat i permet executar independentment cada versió de la pràctica.

Finalment he implementat autenticació bàsica (**Basic Authentication**) amb nginx mitjançant un arxiu `.htpasswd`. Això restringeix l'accés a l'aplicació i afegeix una petita capa de seguretat abans d'accedir als serveis web.

El fet de realitzar totes les proves manualment i documentar-les pas a pas m'ha ajudat a entendre millor el funcionament real de cada component del sistema.